



Nền tảng phát triển Web

Buổi 10:DOM – JQuery



Nội dung

01

Khái niệm DOM

02

DOM Element

03

DOM HTML

04

DOM CSS

05

DOM Events

06

DOM Events Listener

07

DOM Navigation

08

DOM Nodes

01. DOM là gì?

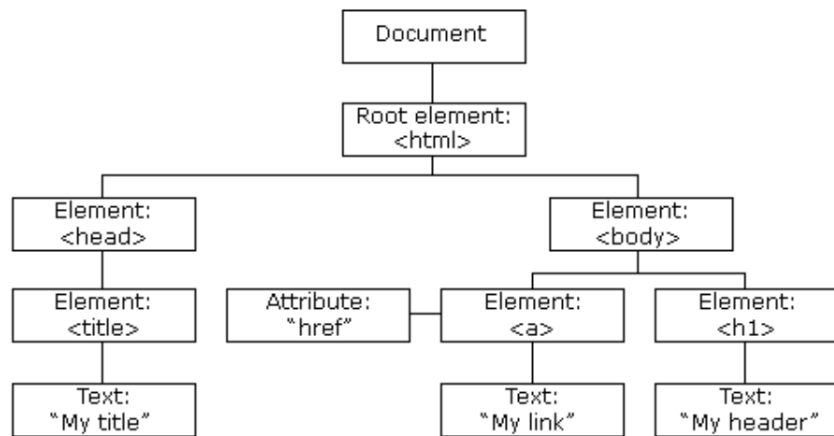
1.1. DOM là gì?

DOM là viết tắt của chữ Document Object Model, dịch ra là mô hình các đối tượng trong tài liệu HTML.

Thông qua mô hình DOM ta có thể truy xuất đến các thẻ HTML một cách dễ dàng.

DOM gồm 3 thành phần: Element, Attribute, Text

The HTML DOM Tree of Objects



01. DOM là gì?

1.2. Nhiệm vụ của DOM Javascript là gì?

- Cho phép truy xuất đến các thẻ html
- Thay đổi các thuộc tính của thẻ html
- Thay đổi css của các thẻ html
- Tạo, xóa, thêm các thẻ html

01. DOM là gì?

1.3. Các loại DOM trong Javascript

- DOM document: có nhiệm vụ lưu trữ toàn bộ các thành phần trong tài liệu của website (Ví dụ bên trên)
- DOM element: có nhiệm vụ truy xuất tới thẻ HTML nào đó thông qua các thuộc tính như tên class, id, name của thẻ HTML
- DOM HTML: có nhiệm vụ thay đổi giá trị nội dung và giá trị thuộc tính của các thẻ HTML
- DOM CSS: có nhiệm vụ thay đổi các định dạng CSS của thẻ HTML
- DOM Event: có nhiệm vụ gán các sự kiện như onclick(), onload() vào các thẻ HTML
- DOM Listener: có nhiệm vụ lắng nghe các sự kiện tác động lên thẻ HTML đó
- DOM Navigation: dùng để quản lý, thao tác với các thẻ HTML, thể hiện mối quan hệ

02. DOM Element

2.1. Sử dụng getElementById để tìm theo ID

Để truy xuất tới một thẻ HTML theo ID ta sử dụng cú pháp sau:

```
var element = document.getElementById('idName');
```

Ví dụ:

```
var element = document.getElementById("idTest");  
element.classList.add("class-test");
```

02. DOM Element

2.2. Sử dụng `getElementsByTagName` để tìm theo tên thẻ html

Tên thẻ HTML chính là tên các thẻ như `p`, `a`, `div`, ... Và ta sẽ truy xuất tới nó bằng cú pháp sau:

```
var element = document.getElementsByTagName('tagName');
```

Kết quả sẽ trả về 1 mảng các object.

Ví dụ:

```
var element = document.getElementsByTagName("p");  
element[0].classList.add("class-test");
```

02. DOM Element

2.3. Sử dụng `getElementsByClassName` để tìm theo tên class

Để tìm các thẻ HTML có class nào đó thì ta dùng cú pháp sau:

```
var element = document.getElementsByClassName('className');
```

Ví dụ:

```
var element = document.getElementsByClassName("p");  
element[0].classList.add("class-test");
```

02. DOM Element

2.4. Sử dụng `querySelector` để tìm theo CSS selector

Để tìm một thẻ HTML theo bộ chọn CSS thì ta dùng cú pháp sau:

```
var element = document.querySelector('selector');
```

Ví dụ:

```
var element = document.querySelector('ul li a');
```

```
element.classList.add("class-test");
```

02. DOM Element

2.5. Sử dụng `querySelectorAll` để tìm theo CSS selector

Để tìm các thẻ HTML theo bộ chọn CSS thì ta dùng cú pháp sau:

```
var element = document.querySelectorAll('selector');
```

Ví dụ:

```
var element = document.querySelectorAll('ul li a');  
element[0].classList.add("class-test");
```

03. DOM HTML

3.1. Lấy nội dung trong thẻ HTML

Để lấy nội dung của một element ta dùng innerHTML:

```
<h2>Nội dung hiện tại</h2>
```

```
document.querySelector("h2").innerHTML;
```

Để thay đổi nội dung của một element ta dùng innerHTML và gán lại giá trị:

```
document.querySelector("h2").innerHTML = "Nội dung mới";
```

03. DOM HTML

3.2. Thay đổi thuộc tính thẻ HTML

Để lấy giá trị của thuộc tính HTML ta sử dụng cú pháp sau:

```
<h2>Nội dung hiện tại</h2>
```

```
document.querySelector("h2").attributeName;
```

Hoặc:

```
document.querySelector("h2").getAttribute("attributeName");
```

Để thay đổi giá trị của thuộc tính HTML thì ta sử dụng cú pháp như sau:

```
document.querySelector("h2").attributeName = "Nội dung mới";
```

Hoặc:

```
document.querySelector("h2").setAttribute("attributeName", "Nội dung mới");
```

04. DOM CSS



Cú pháp thiết lập giá trị:

```
document.getElementById("idName").style.propertyName = 'value';
```

Cú pháp lấy giá trị:

```
document.getElementById("idName").style.propertyName;
```

Trường hợp thuộc tính có dấu gạch ngang như: font-size, line-height, margin-bottom thì nó sẽ có tên là fontSize, lineHeight, marginBottom *(nghĩa là sẽ bỏ đi dấu gạch ngang và viết hoa ký tự đầu tiên của chữ thứ hai)*.

Tài liệu tham khảo thêm: https://www.w3schools.com/jsref/dom_obj_style.asp

01. DOM Events

- **Dom Event** (sự kiện) là một **tác động** nào đó lên các **đối tượng HTML**. Qua đó ta có thể **bắt được sự kiện** và viết một đoạn Javascript để **thực thi một chương trình** nào đó.

- Cú pháp:

```
element.eventname = function() {  
    // Code  
}
```

- Trong đó:
 - **element**: là phần tử muốn bắt sự kiện.
 - **eventname**: Tên sự kiện.

01. DOM Events

- **Danh sách** một số DOM Events phổ biến:

Sự kiện	Mô tả
onload	Khi trình duyệt đã load xong mọi thứ (image, js, css) thì những đoạn code nằm bên trong đó mới được chạy.
onblur	Kích hoạt khi một phần tử mất trọng tâm (không được focus vào nữa)
onfocus	Kích hoạt khi một phần tử hiển thị đúng trọng tâm (được focus vào)
onkeydown	Kích hoạt khi một phím được nhấn
onkeyup	Kích hoạt khi một phím được nhả ra
onclick	Kích hoạt trên con chuột vừa nhấn vào phần tử
onchange	Kích hoạt khi giá trị được thay đổi khác đi so với giá trị trước đó

<code>onclick</code>	Xảy ra khi click vào phần tử HTML
<code>ondblclick</code>	Xảy ra khi double click vào phần tử HTML
<code>onchange</code>	Xảy ra khi giá trị (hay value) của phần tử HTML được thay đổi. Thường dùng trong các đối tượng nhập dữ liệu như form input
<code>onmouseover</code>	Xảy ra khi con trỏ chuột bắt đầu đi vào phần tử HTML
<code>onmouseout</code>	Xảy ra khi con trỏ chuột bắt đầu rời khỏi phần tử HTML
<code>onmouseenter</code>	Tương tự như onmouseover
<code>onmouseleave</code>	Tương tự như onmouseout
<code>onmousemove</code>	Xảy ra khi con chuột di chuyển bên trong phần tử HTML
<code>onkeydown</code>	Xảy ra khi gõ một phím bất kỳ vào ô input
<code>onload</code>	Xảy ra khi phần tử HTML bắt đầu chạy, nó giống như hàm khởi tạo trong lập trình hướng đối tượng vậy đó.
<code>onkeyup</code>	Xảy ra khi bạn gõ phím nhưng lúc bạn bỏ phím ra sẽ được kích hoạt
<code>onkeypress</code>	Xảy ra khi bạn nhấn một phím vào ô input
<code>onblur</code>	Xảy ra khi con trỏ chuột rời khỏi ô input
<code>oncopy</code>	Xảy ra khi bạn copy nội dung của phần tử HTML
<code>oncut</code>	Xảy ra khi bạn cắt nội dung của phần tử HTML
<code>onpaste</code>	Xảy ra khi bạn dán nội dung vào phần tử HTML

02. DOM Events Listener

- **DOM Events Listener** giống Dom Event, nhưng khác ở chỗ:
 - Một **element** có thể gọi được nhiều sự kiện.
 - Có thể **hủy bỏ lắng nghe sự kiện** bất kỳ (*Dom Events chỉ lắng nghe được nhưng không hủy bỏ lắng nghe được*).
- Cú pháp:

```
element.addEventListener("eventname", function(e) {  
    // Code  
});
```
- Trong đó:
 - **element**: là phần tử muốn bắt sự kiện.
 - **eventname**: Tên sự kiện (bỏ chữ **on**, Ví dụ: onclick → click).

03. DOM Navigation

- Dùng để quản lý, thao tác với các thẻ HTML, thể hiện **mối quan hệ cha – con** của **các thẻ HTML**.
- Các thuộc tính:
 - **parentNode**: dùng để truy cập nút cha của nút được chỉ định trong cây DOM
 - **childNodes**: truy cập tất cả các node con của một phần tử nhất định. Node con có thể là một văn bản, chú thích,...
 - **firstElementChild**: trả về phần tử là node con đầu tiên của phần tử cha.
 - **lastElementChild**: trả về phần tử là node con cuối cùng của phần tử cha.
 - **nextElementSibling**: trả về phần tử là node kế tiếp.
 - **previousElementSibling**: trả về phần tử là node trước đó.
 - **nodeName**: Trả về tên một node.

04. DOM Nodes

4.1. document.createElement()

- **Tạo một Node mới** hoàn toàn và không liên quan tới những thẻ HTML đang hiển thị trên website.
- Cú pháp:
`var tenBien = document.createElement("tagName");`
- Trong đó:
 - **tagName**: là tên thẻ. Ví dụ: p, h1, div,...

04. DOM Nodes

4.2. document.createTextNode()

- Text node là một **node đặc biệt**, nó không phải là một thẻ HTML thông thường mà chỉ là một chuỗi (string).
- Cú pháp:

```
var tenBien = document.createTextNode("Nội dung...");
```

04. DOM Nodes

4.3. Phương thức appendChild()

- Dùng để **thêm** (bổ sung) vào **vị trí cuối cùng** của đối tượng một thẻ HTML nào đó.
- Cú pháp:

`element_parent.appendChild(node_insert);`

- Trong đó:
 - **element_parent**: là element cha.
 - **node_insert**: là node bạn muốn thêm vào.

04. DOM Nodes

4.4. Phương thức insertBefore()

- Được dùng để **thêm một Node** vào **đằng trước một node** con nào đó.
- Cú pháp:

element_parent.insertBefore(node_insert, node_child);

- Trong đó:
 - **element_parent**: là element cha.
 - **node_insert**: là node bạn muốn thêm vào.
 - **node_child**: là node con mà bạn muốn thêm node_insert vào đằng trước nó.

04. DOM Nodes

4.5. Phương thức removeChild()

- Được dùng để **xóa một node** con ra khỏi node hiện tại.
- Cú pháp:

element_parent.removeChild(node_remove);

- Trong đó:
 - **element_parent**: là element cha.
 - **node_remove**: là node cần xóa.

04. DOM Nodes

4.6. Phương thức `replaceChild()`

- Dùng để **replace** (thay thế) một **node con** nào đó bằng một **node khác** mới hoàn toàn.
- Cú pháp:

`element_parent.replaceChild(node_insert, node_remove);`

- Trong đó:
 - **`element_parent`**: là element cha.
 - **`node_insert`**: là element mới muốn thay thế.
 - **`node_remove`**: là element cũ muốn bỏ đi.